

无人机街机飞行游戏 3D 小游戏设计文档

面向作业提交与 AI agent 实现的简洁版设计说明

核心体验

操纵无人机飞行，识别威胁区域，保持速度并准确撞击移动目标。

关键词

街机化飞行 / 惯性 / 航线判断 / 锁定压力 / 精确撞击

版本：课程作业设计稿 | 2026

1. 游戏概念

玩家以第三人称 3D 视角操纵一架四轴无人机，在有限边界地图中飞行，目标是撞击一个被明确标记、持续移动的装甲目标。游戏采用偏街机化的飞行模型，不追求真实军事模拟，重点呈现飞行控制、观察威胁、路线修正与最终命中的连续压力。

设计定位

一款小体量、可快速迭代的 3D arcade flight 游戏。每关约 1-2 分钟，规则清楚、反馈强、失败原因可理解。

本设计中的目标与防御单位仅作为游戏对象和机制载体，不包含真实军事操作细节。

玩家体验目标

- 飞行方向与速度具有轻微惯性，形成操作重量感。
- 危险区域边界清晰，玩家能通过声音和 HUD 读懂锁定进度。
- 最终撞击需要保持足够速度，避免变成低速悬停瞄准。
- 失败后能快速重开，鼓励玩家学习路线与节奏。

2. 核心循环与玩家操作

核心循环：观察目标与威胁 -> 选择航线 -> 加速、转向、调整高度 -> 规避锁定或障碍 -> 保持速度接近目标 -> 准确撞击 -> 成功或重开。

输入	功能	设计意图
W / S	加速 / 减速	控制接近速度与转弯余量
鼠标左右	控制航向	主要改变飞行方向
鼠标上下	抬头 / 俯冲，调整高度	用于穿越不同高度的空间
A / D	辅助侧倾 / 提升转弯能力	强化 bank -> turn -> level out 的节奏
Space / Ctrl	上升 / 下降或短时 Boost	提供紧急躲避与路线修正

飞行模型建议

使用简化 arcade flight model：无人机有朝向、速度和惯性；朝向变化不会瞬间改变速度方向，实际速度逐渐趋向目标速度。避免真实空气动力学、复杂姿态模拟与过多按键。

伪代码

```
heading += yawInput * turnRate * dt
desiredVelocity = forwardVector * speed
velocity = lerp(velocity, desiredVelocity, handling * dt)
position += velocity * dt
```

3. 基础系统

系统	职责	最小可行实现
DroneController	输入、速度、朝向、碰撞	刚体或自定义运动；限制最大速度与高度
TargetMover	目标持续移动	沿路径点或简单路线移动，超出边界时回转
DefenseZone	随防御单位移动的威胁范围	触发器 + 半透明圆柱 / 圆圈可视化
LockSystem	进入区域后的渐进锁定	进入加速、离开衰减、满值失败
HUDSystem	目标、距离、锁定、速度、状态	统一数据源，避免逻辑写死在 UI
LevelManager	关卡配置、胜负、重开	数据驱动的关卡参数与对象列表

实现原则

- 先完成单关可玩闭环，再增加障碍、更多防御单位与城市美术。
- 威胁区域、目标标记与边界必须比环境更醒目。
- 所有失败都给出直接原因：锁定完成、撞击障碍、坠地、命中失败。
- 关卡参数尽量配置化，方便 AI agent 或后续开发者调节。

4. 威胁区域与锁定机制

移动防御单位周围存在清晰可见的危险区域。区域会跟随单位移动，玩家进入后开始被探测和锁定。高空不能简单绕过该机制：只要进入区域范围，就会触发威胁。

状态流程

SAFE -> DETECTED -> LOCKING -> INTERCEPTED

阶段	表现	规则
Safe	无提示	未进入任何威胁区域
Detected	区域变亮，出现短促提示音	进入防御区域，开始累计锁定
Locking	LOCK 进度条、报警加快、屏幕轻微强调	锁定值随时间增加；离开后逐渐降低
Intercepted	警报达到峰值，立即失败	锁定值达到 100%

可读性要求

红色半透明区域 + 轮廓边界 + 地面投影 + 声音节奏 + HUD
进度条同时使用。玩家应在一眼内知道哪里危险、自己还有多少时间。

建议参数

$lockValue += lockRate * dt$ ；离开区域后 $lockValue -= decayRate * dt$ ； $lockValue = clamp(lockValue, 0, 100)$ 。不同防御单位可拥有不同范围、锁定速度与移动速度。

5. Level 1 / 2 / 3 设计

关卡	场景与对象	主要挑战
Level 1 Open Field	空旷、有限边界地图；1 个移动目标；少量移动防御单位；几乎没有实体障碍。	学习飞行、读懂危险区域、脱离锁定，并以足够速度撞击移动目标。
Level 2 Obstructed Field	更多防御单位；危险区域可能重叠；加入岩石、墙壁、塔或残骸。	需要更精确地转弯、修正高度与速度，同时避免锁定和撞击障碍。
Level 3 Urban Environment	城市环境；建筑、街道、高墙、窄缝与大量遮挡；保留移动防御单位和目标。	可用空间进一步缩小，需要在城市几何空间中连续决策和快速补救。

递进逻辑

Level 1 增加规则理解成本；Level 2 增加空间障碍与威胁密度；Level 3 压缩航线空间并提高连续操作要求。三关不依赖全新操作，而是逐步提高同一套核心能力的精度与压力。

6. 目标自身防御、HUD 与胜负

目标自身防御

移动目标附近存在较小的近距离防御范围。玩家进入范围后，如果速度低于最低撞击速度，会被击落。这样最终阶段不能依靠悬停慢慢调整，而要在正确路线、角度和速度下完成撞击。

HUD 信息层级

区域	显示	用途
中央 / 前方	目标透视标记、TARGET、距离	持续追踪移动目标，即使被遮挡也不迷失
顶部或中央	LOCK 进度条、状态文字	表达威胁紧迫度
边角	速度、高度、边界警告	帮助玩家判断能否安全撞击
全屏反馈	警报音、闪烁、轻微震动	让状态变化无需阅读大量文字

胜利条件

无人机以有效速度与目标发生有效碰撞，目标被摧毁，显示关卡完成并允许进入下一关。

失败条件

锁定值达到
100%；撞击地面、建筑或其他实体障碍；进入目标近距离防御范围但速度不足；撞击未命中目标并坠落。

7. 对象与数据结构建议

建议采用数据驱动方式，让对象逻辑与关卡配置分离。以下结构足以支持原型开发，也便于 AI agent 分模块生成代码。

对象 / 数据	关键字段
DroneConfig	maxSpeed, acceleration, handling, turnRate, minImpactSpeed, boostDuration
TargetConfig	pathPoints, moveSpeed, defenseRadius, requiredImpactSpeed
DefenseUnitConfig	position, movePath, zoneRadius, lockRate, decayRate, heightRange
ObstacleConfig	position, rotation, size, collisionType, visualPrefab
LevelConfig	boundary, spawnPoint, target, defenses[], obstacles[], skyHeight
GameState	Playing, Won, Failed, Restarting
LockState	Safe, Detected, Locking, Intercepted; value 0-100

模块边界建议

DroneController 只负责飞行；ThreatZone 只负责检测范围；LockSystem 只负责锁定数值和状态；HUD 订阅状态变化；LevelManager 负责胜负与重开。模块之间通过事件或清晰接口通信，避免所有逻辑集中到一个脚本。

8. 实现建议与开发顺序

推荐开发顺序

- 建立有限边界与第三人称跟随镜头。
- 实现无人机 arcade 飞行：朝向、惯性、速度、上升下降与碰撞。
- 加入单个移动目标和透视标记，完成一次有效撞击。
- 加入一个可移动威胁区域、锁定进度、报警与失败重开。
- 制作 Level 1，完成从开始到胜利的完整闭环。
- 复制关卡配置制作 Level 2：增加防御单位与障碍。
- 制作 Level 3：使用模块化城市障碍组合空间压力。
- 最后统一 HUD、音效、视觉反馈、参数平衡与教学提示。

平衡与可玩性检查

- 玩家应能理解自己为什么失败，而不是感觉随机死亡。
- 危险区域边界不能被场景颜色吞没。
- 目标标记要稳定、明显，但不能替玩家自动导航。
- 锁定时间应留出反应窗口，同时要求玩家提前规划。
- 目标防御范围应鼓励高速命中，不应让玩家只能碰运气。
- 每关重开速度要快，失败成本低，便于反复尝试。

交付标准

可运行的最小版本应包含：第三人称无人机控制、有限地图、移动目标、至少一个移动威胁区、锁定与报警、障碍碰撞、胜负状态、HUD 和快速重开。

一句话总结

这不是上下移动的跑酷，而是一款围绕惯性、承诺与风险补救展开的街机化 3D 飞行游戏。